

05 设计模式与软件构造复习讲义

目录

5. 设计模式	2
5.1 设计模式的构成	2
5.2 Strategy 策略模式	2
5.3 Abstract Factory 抽象工厂	4
5.4 Factory Method 工厂方法	5
5.5 Singleton 单例	6
5.6 Iterator 迭代器	6
6. 代码设计与软件构造	7
6.1 代码设计关注点	7
6.2 布局	8
6.3 命名	8
6.4 注释与 Javadoc	9
6.5 AI 代码生成 Prompt	10
6.6 软件构造	10
6.7 构造技术	11
6.8 代码审查	11
6.9 集成与构建	12
6.10 构造管理	12
6.11 坏味道与重构	13
6.12 TDD	14

目录	2
6.13 AI 编程原则	14
对应考试模板	15
7.9 设计模式题模板	15
7.10 代码质量题模板	15
快速背诵清单	16
设计模式	16
构造	16

范围：设计模式.pdf、代码设计.pdf、软件构造.pdf；结合 Exam/Review 中策略模式、抽象工厂、代码质量、重构、审查、测试和构造管理内容。结构：先整理 PPT 主线，再放对应考试模板和快速背诵清单。

5. 设计模式

5.1 设计模式的构成

一个设计模式通常包含：

- 名称
- 问题
- 解决方案
- 应用场景
- 注意事项

模式不是为了“套模板”，而是把稳定抽象和变化点分开。

5.2 Strategy 策略模式

定义：

定义一组算法，把每个算法封装起来，并让它们可以互换，使算法变化独立于使用算法的客户。

结构：

- Context：持有 Strategy。
- Strategy：算法接口。
- ConcreteStrategy：具体算法。

适用场景：

- 同一业务存在多种算法。
- 需要运行时切换算法。
- 代码中存在大量按类型分支选择行为。

例子：充值手续费策略

```
public interface FeeStrategy {
    BigDecimal calculateFee(BigDecimal amount);
}

public class BankCardFeeStrategy implements FeeStrategy {
    public BigDecimal calculateFee(BigDecimal amount) {
        return amount.multiply(new BigDecimal("0.005"));
    }
}

public class AlipayFeeStrategy implements FeeStrategy {
    public BigDecimal calculateFee(BigDecimal amount) {
        return BigDecimal.ZERO;
    }
}
```

Context:

```
public class RechargeServiceImpl implements RechargeService {
    private final FeeStrategy feeStrategy;

    public RechargeServiceImpl(FeeStrategy feeStrategy) {
        this.feeStrategy = feeStrategy;
    }

    public void recharge(BigDecimal amount) {
        BigDecimal fee = feeStrategy.calculateFee(amount);
        // 执行业务流程
    }
}
```

优点:

- 消除复杂条件分支。
- 新增算法时新增策略类。

- 符合 OCP。

代价：

- 类数量增加。
- 客户端需要选择策略。
- Strategy 与 Context 之间需要约定输入输出。

5.3 Abstract Factory 抽象工厂

定义：

提供一个创建一组相关或相互依赖对象的接口，而不指定它们的具体类。

结构：

- AbstractFactory
- ConcreteFactory
- AbstractProduct
- ConcreteProduct
- Client

适用场景：

- 系统需要创建一组相关产品。
- 产品族需要整体替换。
- 客户端不应依赖具体产品类。

例子：不同平台 UI 组件族

```
public interface UIFactory {  
    Button createButton();  
    Dialog createDialog();  
}
```

```
public class WebUIFactory implements UIFactory {  
    public Button createButton() {  
        return new WebButton();  
    }  
  
    public Dialog createDialog() {  
        return new WebDialog();  
    }  
}
```

优点:

- 保证同一产品族一起使用。
- 客户端依赖抽象。
- 切换产品族集中在工厂。

缺点:

- 新增产品种类会修改工厂接口。
- 产品族稳定时效果最好。

5.4 Factory Method 工厂方法

定义:

父类定义创建对象的接口，由子类决定实例化哪一个具体类。

适用场景:

- 父类无法提前确定要创建的具体对象。
- 创建逻辑要延迟到子类。

与抽象工厂区别:

- 工厂方法关注创建一个产品。
- 抽象工厂关注创建一组相关产品。

5.5 Singleton 单例

定义：

保证一个类只有一个实例，并提供全局访问点。

使用时要谨慎：

- 单例容易形成全局状态。
- 会增加隐式耦合。
- 测试替换不方便。

适合：

- 无状态工具入口。
- 全局唯一资源管理器。
- 配置对象。

不适合：

- 承载业务状态。
- 替代依赖注入。

5.6 Iterator 迭代器

定义：

在不暴露聚合对象内部表示的前提下，顺序访问其中元素。

结构：

- Iterator
- ConcreteIterator
- Aggregate

- ConcreteAggregate

优点:

- 隐藏集合内部结构。
- 简化聚合对象接口。
- 支持多种遍历方式。
- 支持多个遍历同时进行。

例子:

客户端使用迭代器遍历充值记录，不关心底层是数组、链表、数据库游标还是分页结果。

6. 代码设计与软件构造

6.1 代码设计关注点

代码设计关注可读性、可维护性和可验证性。

主要内容:

- 布局
- 命名
- 注释
- Javadoc
- 小任务拆分
- 复杂决策表达
- 数据使用
- 显式依赖
- 断言

6.2 布局

原则：

- 缩进表达逻辑层级。
- 空行分隔不同语义块。
- 长表达式拆行。
- 相似语句对齐。
- 复杂控制结构保持清楚。

坏布局会让正确代码难以审查。

6.3 命名

命名要求：

- 表达含义。
- 与领域术语一致。
- 避免误导。
- 避免无意义缩写。
- 避免过长。

Java 约定：

- 包名：小写单词。
- 类名和接口名：PascalCase。
- 变量和方法：camelCase。
- 常量：大写字母加下划线。

语义规则：

- 类、属性、数据对象用名词。

- 接口可用名词或形容词。
- 方法用动词或动词加名词。

示例：

- RechargeService
- CampusCardRepository
- findRechargeRecordsByCardNo
- MAX_RECHARGE_AMOUNT

6.4 注释与 Javadoc

Java 注释：

- `//`：单行注释。
- `/* */`：普通块注释。
- `/** */`：文档注释。

Javadoc 常用标签：

- `@author`
- `@version`
- `@see`
- `@since`
- `@deprecated`
- `@param`
- `@return`
- `@throws`

注释原则：

- 解释意图、约束和不直观决策。
- 不重复代码表面含义。
- 公共 API 要说明参数、返回值和异常。

6.5 AI 代码生成 Prompt

四个元素：

- Context：上下文。
- Goal：目标。
- Format：输出格式。
- Steps：步骤。

示例：

Context：

项目使用 `Spring Boot`，分层为 `controller/service/repository`。

Goal：

实现校园卡充值服务接口和实现类。

Format：

输出 `Java` 接口、实现类和单元测试。

Steps：

先定义 `DTO`，再定义 `Service` 接口，再写实现逻辑，最后写测试。

6.6 软件构造

SWEBOK 中软件构造覆盖：

- 编码
- 验证
- 单元测试
- 集成测试
- 调试

构造不是“写代码”一个动作，而是把设计转化为可运行、可验证、可维护的软件制品。

6.7 构造技术

主要技术：

- 可理解的源代码
- 类、枚举、变量、常量
- 控制结构
- 错误处理
- 代码级安全
- 单元测试
- 集成测试
- 调试

6.8 代码审查

课程给出的审查经验：

- 每次审查少于 200 到 400 行代码。
- 审查速度控制在每小时 300 到 500 行。
- 单次审查时间控制在 60 到 90 分钟。
- 开发者先对代码做注解。
- 使用检查表。
- 确认缺陷已修复。
- 保持积极的审查文化。
- 借助工具支持轻量化审查。

AI 辅助代码审查适合：

- 识别坏味道。
- 检查安全问题。

- 检查风格问题。
- 发现遗漏测试。

最终结论要由人结合需求、设计和上下文确认。

6.9 集成与构建

集成策略：

- 大爆炸集成：一次性集成所有模块。
- 增量集成：逐步集成并验证模块。

增量集成更利于定位问题和控制风险。

构建：

将源代码、资源、配置和依赖转化为可执行或可部署制品。

构建依赖配置管理：

- 版本一致。
- 依赖可追踪。
- 构建脚本受控。
- 发布制品可复现。

6.10 构造管理

构造管理内容：

- 分配构造任务。
- 跟踪代码进度。
- 度量复杂度。
- 度量代码行数。
- 度量注释比例。
- 管理分支、合并和发布。
- 执行代码审查和测试门禁。

6.11 坏味道与重构

重构定义：

在不改变软件外部行为的前提下，改善内部结构。

常见坏味道：

- 重复代码。
- 长方法。
- 大类。
- 长参数列表。
- Magic Number 或 Magic String。
- 空 catch。
- 过多注释。
- 某个类过度访问其他类属性。
- Switch 或 if-else 按类型分发行为。

对应改进：

坏味道	问题	重构
重复代码	隐式耦合	提取方法或提取类
长方法	难理解、难测试	提取方法
大类	职责过多	提取类、按职责拆分
长参数列表	调用复杂	引入参数对象
Magic Number	含义不清	提取常量
空 catch	吞掉错误	记录日志或抛出业务异常
过多注释	结构表达力不足	改名、拆分、重组逻辑
过度访问其他类属性	职责位置错误	移动方法或委托
类型分支	违反 OCP	策略模式或多态

6.12 TDD

TDD 循环：

1. Red：先写失败测试。
2. Green：写最少代码让测试通过。
3. Refactor：在测试保护下重构。

适合：

- 规则明确的业务逻辑。
- 算法。
- 服务层。
- 状态迁移。

6.13 AI 编程原则

使用 AI 写代码时：

- 明确意图。
- 逐步细化。
- 保持对代码的理解。
- 先定义测试或验收标准。
- 经过验证后再提交。

不能把 AI 输出直接当作正确实现。需要用需求、设计、测试和代码审查约束它。

对应考试模板

7.9 设计模式题模板

题目出现大量按类型选择算法：

```
if (payType.equals("BANK")) { ... }  
else if (payType.equals("ALIPAY")) { ... }  
else if (payType.equals("WECHAT")) { ... }
```

答：

- 使用 Strategy。
- 抽象 PaymentStrategy。
- 每种支付方式一个具体策略。
- RechargeService 依赖 PaymentStrategy 或策略工厂。
- 新增支付方式时新增策略类，减少修改原业务流程。

题目出现一组相关产品整体替换：

- 使用 Abstract Factory。
- 抽象产品和抽象工厂。
- 具体工厂创建同一产品族。

题目要求隐藏集合结构：

- 使用 Iterator。
- 客户端通过迭代器访问元素。

7.10 代码质量题模板

答题结构：

问题：

违反原则：

影响：

重构：

测试：

示例：

问题：充值方法过长，同时处理参数校验、支付调用、余额更新、记录保存和异常处理。

违反原则：SRP、SOFA。

影响：代码难理解、难测试、修改支付逻辑会影响余额更新逻辑。

重构：提取校验方法、支付网关、余额更新方法和记录保存方法；支付方式变化使用 Strategy。

测试：为正常充值、支付失败、校园卡挂失、金额非法分别编写单元测试。

快速背诵清单

设计模式

- Strategy：封装可互换算法。
- Abstract Factory：创建一组相关产品。
- Factory Method：子类决定创建哪个产品。
- Singleton：唯一实例和全局访问点。
- Iterator：隐藏集合内部结构并顺序访问元素。

构造

- 构造 = 编码 + 验证 + 单元测试 + 集成测试 + 调试。
- 代码审查要小批量、限时、用检查表。
- 重构不改变外部行为，只改善内部结构。
- TDD：Red、Green、Refactor。